

Fault Localization

2025/2026

João Saraiva

***Departamento de Informática
Universidade do Minho,
Portugal***

Motivation: Where is the Fault?

```
1  /* Programa que da o maior de tres numeros lidos do teclado */
2
3  #include <stdio.h>
4  int main()
5  { int a,b,c,m;
6
7      printf("Introduza 3 números:\n");
8      scanf("%d%d%d",&a,&b,&c);
9
10     if (! (a > b))
11     { if (b > c)
12         m = b;
13         else m = c; }
14     else
15     { if ( a > c)
16         m = a;
17         else m = b; }
18
19     printf("\n O maior e: %d\n",m);
20     return 0;
21 }
```

Motivation: Where is the Fault?

Hum... **Code Smells** make program comprehension hard!

Motivation: Where is the Fault?

```
1  /* Programa que da o maior de tres numeros lidos do teclado */
2
3  #include <stdio.h>
4  int main()
5  { int a,b,c,m;
6
7      printf("Introduza 3 números:\n");
8      scanf("%d%d%d",&a,&b,&c);
9
10     if ( a > b)
11     { if ( a > c)
12         m = a;
13         else m = b; }
14     else
15     { if (b > c)
16         m = b;
17         else m = c; }
18     printf("\n O maior e: %d\n",m);
19     return 0;
20 }
```

How did we solve the problem?

- Abstracting from the code: **using logic** reasoning to check whether all numbers were compared (between them)
- Or, we **consider (some) inputs** and check whether the result is the correct. As soon we find one fault, we analyze the code executed on that input!

Spectrum-based Fault Localization

It mimics the second approach:

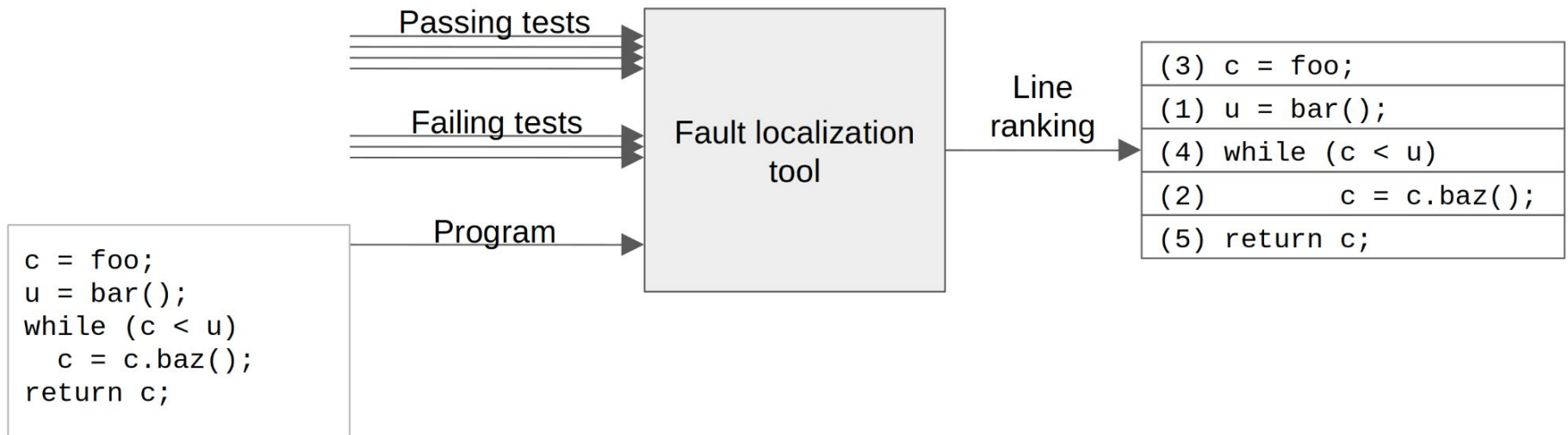
- It executes the program with (available) test cases
- It registers which source code components were used when executing the test cases

Code components: methods/functions, modules, program lines

- It uses a statistical method to assign probabilities of being faulty to such components.

Fault Localization

Fault localization as a black box



Spectrum-based Fault Localization

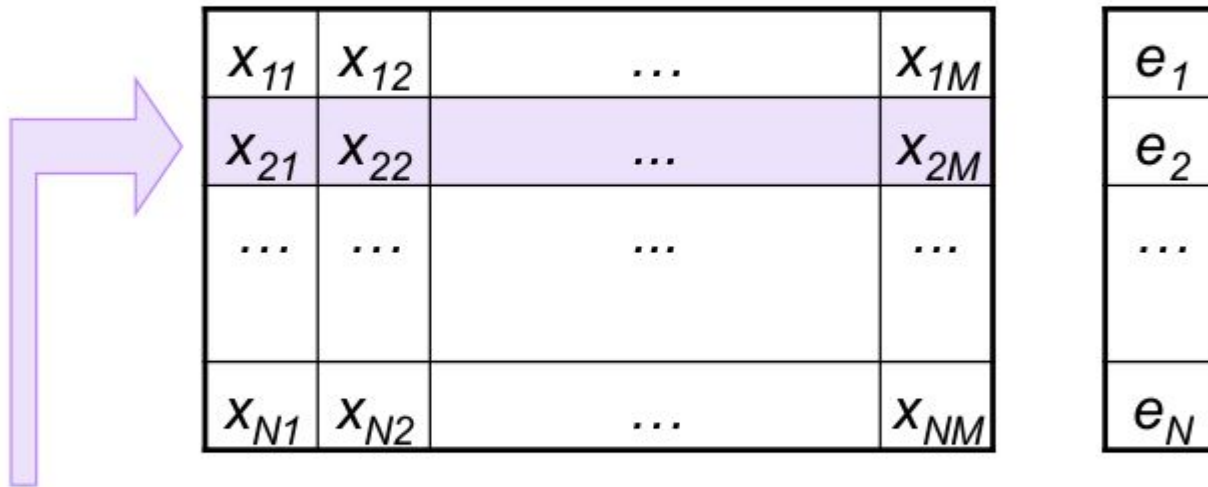
M components

N cases

x_{11}	x_{12}	...	x_{1M}
x_{21}	x_{22}	...	x_{2M}
...	...	...	...
x_{N1}	x_{N2}	...	x_{NM}

Spectrum-based Fault Localization

1. Spectra for M test cases



Row i : the blocks that are executed in case i

Spectrum-based Fault Localization

x_{11}	x_{12}	...	x_{1M}
x_{21}	x_{22}	...	x_{2M}
...	...	...	...
x_{N1}	x_{N2}	...	x_{NM}

e_1
e_2
...
e_N



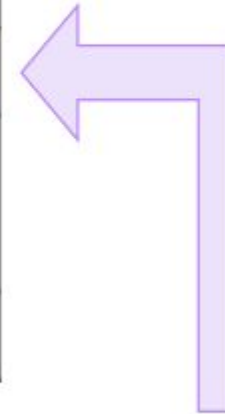
Column j : the test cases in which block j was executed

Spectrum-based Fault Localization

1. Spectra for M test cases
2. Error detection per test case

x_{11}	x_{12}	...	x_{1M}
x_{21}	x_{22}	...	x_{2M}
...	...	...	...
x_{N1}	x_{N2}	...	x_{NM}

e_1
e_2
...
e_N

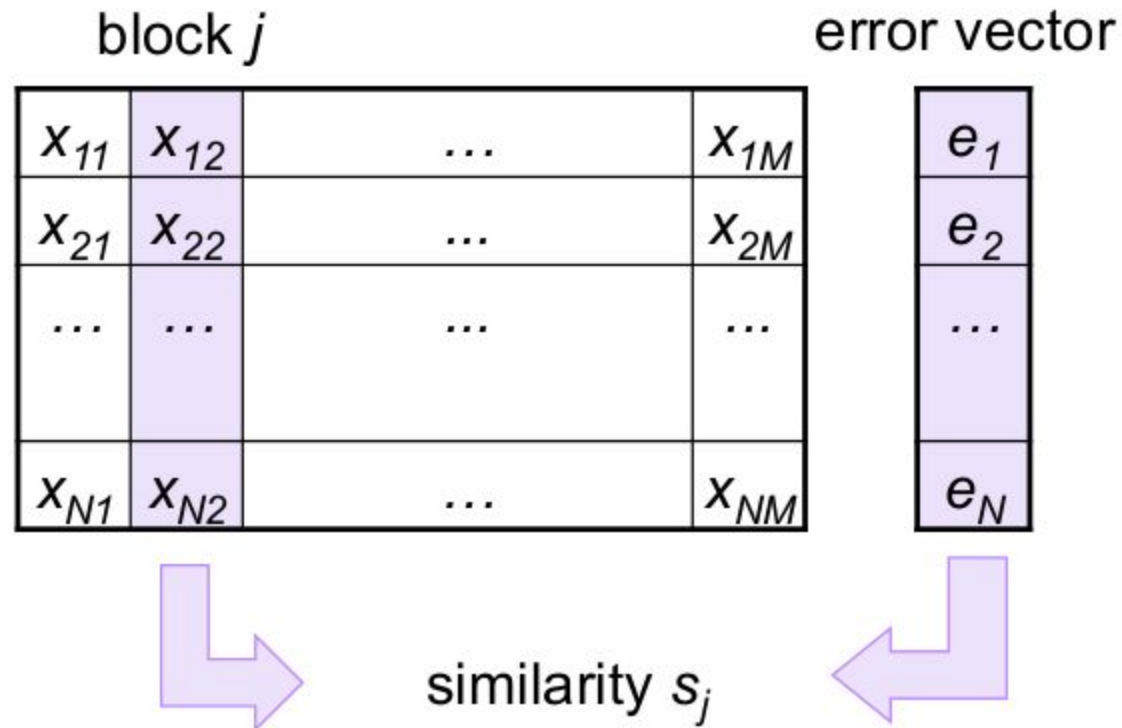


$e_i=1$: error in the i -th test

$e_i=0$: no error in the i -th test

Spectrum-based Fault Localization

Compare every column vector with the error vector.



Spectrum-based Fault Localization

Jaccard similarity coefficient:

block j

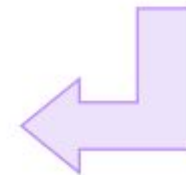
1
0
1
0
1

error vector

0
1
1
0
1



$$s_j = \frac{n_{11}}{n_{11} + n_{10} + n_{01}}$$



Spectrum-based Fault Localization

Jacard Similarity Coefficient

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}}.$$

M_{11} represents the total number of attributes where A and B both have a value of 1.

M_{01} represents the total number of attributes where the attribute of A is 0 and the attribute of B is 1.

M_{10} represents the total number of attributes where the attribute of A is 1 and the attribute of B is 0.

M_{00} represents the total number of attributes where A and B both have a value of 0.

Spectrum-based Fault Localization

Jacard Similarity Coefficient

The Jaccard coefficient is computed as follows:

For each component i :

$$J(i) = \frac{a}{a + b + c}$$

where:

- a = number of test cases that fail and execute component i
- b = number of test cases that pass and execute component i
- c = number of test cases that fail that **do not** execute component i

Spectrum-based Fault Localization

$$s = \frac{n_{11}}{n_{11} + n_{10} + n_{01}}$$

component

	<i>a</i>	<i>b</i>	<i>c</i>	<i>d</i>	<i>e</i>	<i>f</i>	<i>g</i>	<i>fail</i>
<i>test 1</i>	0	1	1	1	1	0	0	0
<i>test 2</i>	0	0	0	1	0	1	1	1
<i>test 3</i>	1	1	1	1	0	0	0	1
<i>test 4</i>	0	0	0	0	0	0	0	0
<i>test 5</i>	1	1	0	1	1	0	1	1
	$\frac{2}{3}$	$\frac{1}{2}$	$\frac{1}{4}$	$\frac{3}{4}$	$\frac{1}{4}$	$\frac{1}{3}$	$\frac{2}{3}$	

Spectrum-based Fault Localization

Components	Worker.c						Tests				
							t1	t2	t3	t4	t5
	1	#include	<stdio.h>								
	2										
	3	int	main(int argc, const char* argv[]) {								
	4	int	factorialTotal = 0;				1	1	1	1	1
	5										
	6	for	(int i = 1; i < argc; i++) {								
	7	int	value = atoi(argv[i]);				1	1	1	1	1
	8	if	(value % 2 == 0) {				1	1	1	1	1
	9	int	factorial = factorialCalculate(value);				1	0	1	0	1
	10		factorialTotal += factorial;				1	0	1	0	1
	11	}									
	12	}									
	13										
	14	printf	("The sum of all numbers' factorial is: %d", factorialTotal);				1	1	1	1	1
	15	}									
							1	0	1	0	1

0.3

0.3

0.3

0.7

0.7

0.3

Spectrum-based Fault Localization

An Evaluation of Similarity Coefficients for Software Fault Localization*

Jacard Similarity Coefficient:

It is simple to compute and it works very well when there is a single fault in the software system under analysis.

However, there are more powerful coefficients, that handle multiple faults, like for example Ochiai.

Rui Abreu

Peter Zoetewij

Arjan J.C. van Gemund

Software Technology Department
Faculty of Electrical Engineering, Mathematics, and Computer Science
Delft University of Technology
P.O. Box 5031, NL-2600 GA Delft, The Netherlands
{r.f.abreu, p.zoetewij, a.j.c.vangemund}@tudelft.nl

Abstract

Automated diagnosis of software faults can improve the efficiency of the debugging process, and is therefore an important technique for the development of dependable software. In this paper we study different similarity coefficients that are applied in the context of a program spectral approach to software fault localization (single programming mistakes). The coefficients studied are taken from the systems diagnosis / automated debugging tools Pinpoint, Tarantula, and AMPLE, and from the molecular biology domain (the Ochiai coefficient). We evaluate these coefficients on the Siemens Suite of benchmark faults, and assess their effectiveness in terms of the position of the actual fault in the probability ranking of fault candidates produced by the diagnosis technique. Our experiments indicate that the Ochiai coefficient consistently outperforms the coefficients currently used by the tools mentioned. In terms of the amount of code that needs to be inspected, this coefficient improves 5% on average over the next best technique, and up to 30% in specific cases.

Keywords: Software reliability, automated debugging, software fault diagnosis, fault localization, program spectra.

1 Introduction

Software reliability can generally be improved through extensive testing and debugging, but this is often in conflict with market conditions: software cannot be tested exhaustively, and of the bugs that are found, only those with the highest impact on the user-perceived reliability can be

solved before the release. In this typical scenario, testing reveals more bugs than can be solved, and debugging is the bottleneck for improving reliability. Automated debugging techniques can help to reduce this bottleneck. These techniques give a diagnosis for errors that are detected during the execution of a program, which can help programmers to locate their root causes, and thus to reduce the effort spent on manual debugging.

Diagnosis techniques, which include automated debugging, can be classified as white box or black box, depending on the amount of knowledge that is required about the system under study. An example of a white box technique is model-based diagnosis (see, e.g., [6, 7]), where a diagnosis is obtained by logical inference from a formal model of the system, combined with a set of run-time observations. While white box approaches to software diagnosis exist (see, e.g., [14, 17, 18, 19]), software modeling is extremely complex. Hence, most software diagnosis techniques are black box. The subject of this paper is a specific automated debugging technique, namely software fault localization through the analysis of program spectra. Because this technique requires practically no information about the system being diagnosed, it can be classified as a black box diagnosis technique.

Program spectra can be seen as projections of traces of software activity. Typically, this projection is much more compact than a trace, which makes program spectra an attractive technique in resource-constrained environments, such as embedded systems. The technique studied in this paper is based on analyzing the differences between program spectra obtained for correct behavior of the software, and program spectra obtained for faulty behavior of the software. An essential part of this analysis is the computation of a measure of similarity between different vectors in the program spectra data and a vector that contains information

*This work has been carried out as part of the TRADER project under the responsibility of the Embedded Systems Institute. This project is partially supported by the Netherlands Ministry of Economic Affairs under the BSIK03021 program.

Spectrum-based Fault Localization

Ochiai coefficient

$$s_O(j) = \frac{n_{11}(j)}{\sqrt{(n_{11}(j) + n_{01}(j)) \times (n_{11}(j) + n_{10}(j))}}$$

Spectrum-based Fault Localization

Ochiai Similarity Coefficient

The Ochiai coefficient for each program's component i computed as follows

For each component i :

$$O(i) = \frac{a}{\sqrt{(a+b)(a+c)}}$$

where:

- a = number of test cases that fail and execute component i
- b = number of test cases that pass and execute component i
- c = number of test cases that fail that **do not** execute component i

Spectrum-based Fault Localization

There are many variants on spectrum-based FL:

Ochiai^[1]

$$S(s) = \frac{failed(s)}{\sqrt{totalfailed \cdot (failed(s) + passed(s))}}$$

Tarantula^[2]

$$S(s) = \frac{failed(s)/totalfailed}{failed(s)/totalfailed + passed(s)/totalpassed}$$

D*^[3]

$$S(s) = \frac{failed(s)^*}{passed(s) + (totalfailed - failed(s))}$$

[1] R. Abreu, P. Zoetewij, and A. J. C. van Gemund. An evaluation of similarity coefficients for software fault localization.

[2] J. Jones, M. J. Harrold, and J. Stasko. Visualization of test information to assist fault localization.

[3] W. E. Wong, V. Debroy, R. Gao, and Y. Li. The DStar method for effective software fault localization.

Motivation: Where is the Fault?

Exercise 1: Using SFL locate the fault in the source code of the following C program. Assume that the sw components correspond to the code in lines 10 to 18.

```
1  /* Programa que da o maior de tres numeros lidos do teclado */
2
3  #include <stdio.h>
4  int main()
5  { int a,b,c,m;
6
7      printf("Introduza 3 números:\n");
8      scanf("%d%d%d",&a,&b,&c);
9
10     if ( a > b)
11         { if ( a > c)
12             m = a;
13             else m = b; }
14     else
15         { if (b > c)
16             m = b;
17             else m = c; }
18     printf("\n O maior e: %d\n",m);
19     return 0;
20 }
```

Motivation: Where is the Fault?

Exercise 2: A sw developer suspects that there is a bug in one of the methods in his program. We ran the program with 6 test cases and he traced which methods were used in the different executions. He also registered whether the test passed (“Passou”) or not (“Falhou”). What can you say about the coverage of these tests? How can you help the developer locating the bug?

		Teste suite					
		T ₁	T ₂	T ₃	T ₄	T ₅	T ₆
Componentes do SW: métodos	m ₁	✓	✓	✓	✓	✓	✓
	m ₂		✓		✓	✓	✓
	m ₃	✓		✓	✓		✓
	m ₅	✓		✓		✓	
	m ₆		✓		✓		✓
	m ₇						
		Passou	Passou	Falhou	Passou	Falhou	Passou

Spectrum-based Fault Localization

Gzoltar: A SFL-based tool for Java

- Source code instrumentation
 - line, method, class, package
- Test case execution
 - Junit testing framework